第24章 單源最短路徑：詳解

24-E01 原地與同步不能混稱同一狀態

推導

以(s,a,b,c,t,z)順序，初始(0,∞,∞,∞,∞,∞)。第1輪sa使a4(parent s)、sb使b1(parent s)，其餘尚無有限起點。第2輪at使t12(parent a)、ac使c5(parent a)、ba使a−1(parent b)。第3輪ct先使t8(parent c)，at再使t7(parent a)，ac使c0(parent a)。第4輪ct使t3(parent c)。第5輪完全無更新。

輪末依序為(0,4,1,∞,∞,∞)、(0,−1,1,5,12,∞)、(0,−1,1,0,7,∞)、(0,−1,1,0,3,∞)，第5輪同第4輪。最終樹s→b→a→c→t。

若改順序sa、sb、ba、ac、ct、at、bc、bt，第1輪便以剛得到的b1更新a−1，再更新c0、t3，得到最終值。同步DP第1輪只能讀D_0，故只有a4、b1；兩者皆是一輪掃邊，但依賴的舊／新狀態不同。

結論與檢核

原地每輪「至少捕捉該邊數的路」並非「只能使用該邊數」；同步D_i才精確定義最多i邊。原指定順序第5輪無更新，可早停。

24-E02 負環證書與影響範圍

推導

a→c→a權1−2=−1，由s→b→a可進入。s→b→a→(c→a重複r次)→c→t的成本1−2−r+1+3=3−r，任意小，故a,c,t皆−∞。s沒有由cycle返回的路，δ(s,s)=0；b只有s直接進入，δ(s,b)=1；z,p,q都不可達，距離∞。八項為(0,−∞,1,−∞,−∞,∞,∞,∞)。

令$B=\{v:D_n[v]<D_{n-1}[v]\}$。seed的n邊最優walk若沒有負cycle可刪，去除非負cycles得到≤n−1邊且不增成本，矛盾，所以seed確受負cycle影響。其向前後代也受影響。反向，任何可達負cycle全部頂點在D_{n-1}有限；若它上面沒seed，每條u→v都有$D_{n-1}[v]\le D_{n-1}[u]+w$，相加變0≤負總權，矛盾。因此每個可達負cycle至少有一seed，向前搜尋便涵蓋全部受影響點。

結論與檢核

pq圈雖負但沒有s入口；全圖任意負圈偵測應另外加super-source，不能改變此題單源的可達性語義。標記−∞的方向是由圈向後，不是所有能走入圈的祖先。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

24-E03 最短路徑與關鍵路徑的算例

推導

原圖的合法拓撲序s,b,a,c,t,z，所有arc均由前往後。處理s後a4,b1；處理b後a−1,c6,t13；處理a後c0,t7；處理c後t3；t與z無出邊不改。結果(0,−1,1,0,3,∞)。負邊ba不造成問題，因無回邊cycle。

工作最早開始S[A]=S[B]=0，F[A]=3、F[B]=2。S[C]=max(3,2)=3、F[C]=7，S[D]=7、F[D]=8。Critical path為A,C,D，成本3+4+1=8；總工量3+2+4+1=10。

結論與檢核

無限處理器可讓A、B同時開始，8可達。一顆處理器至少須做10單位工，8不能達；例如A[0,3]、B[3,5]、C[5,9]、D[9,10]可在10完成。先後關係下界與資源下界要分開。

24-E04 Dijkstra 的不可重開條件

推導

非負版先取s0，候選a4,b1；取b1後a3(parent b)、c6(parent b)、t13(parent b)；取a3後c4(parent a)、t11(parent a)；取c4後t7(parent c)；取t7，餘z∞停止。已取距離序0,1,3,4,7，最終parent形成s-b-a-c-t。第一次發現t13尚有比13小的候選可改善它，必須等extract t7。

反例取s0後a2,b5；取a2得t4；取t4後才輪b5。ba−4讓真正a=1，接at2得t=3，但a及t已固定。最短s-b-a的中間點b距離5大於終點a距離1，故「最短路後綴非負⇒δ(s,b)≤δ(s,a)」失敗。

結論與檢核

反例是一張DAG，故無負cycle。錯誤來自不滿足Dijkstra非負權契約，不是「負環才使Dijkstra錯」。可改用線性DAG算法求a1,t3。

24-E05 差分限制的兩個證書

推導

邊1→2:−2、2→3:−1、3→1:4。全零依序掃，得x=(0,−2,−3)且不再下降。三個差為−2、−1、3，均不超過−2、−1、4；加3後(3,1,0)最小座標0且差不變。

末常數改2時相加：$(x_2 - x_1) + (x_3 - x_2) + (x_1 - x_3) = 0$，右端−2−1+2=−1，得0≤−1，這是沒有任何可行解的證書。等式$x_1 - x_3 = 3$拆成3→1:3與1→3:−3；原可行向量恰滿足它。

第24章 單源最短路徑：詳解 （續）

整數題中右端逐個floor為−2、−1、2，因左端都是整數，所以等價；已得到相同負cycle矛盾，故整數無解。實數卻可有解，例如(0,−1.2,−1.4)，三差為−1.2、−0.2、1.4≤2.8；整數要求確實改變可行性。

結論與檢核

原常數可行且解可平移；末常數2或全變數整數的改題均無解。方向由被減變數指向正號變數，正負號不可交換。

24-E06 零權前驅與獨立驗證

推導

parent[a]=b、parent[b]=a形成a↔b，從s沿parent children無法到達a,b；雖每邊d[v]=d[parent]+0，卻不能提供s-v路。合法選parent[a]=s、parent[b]=a，均距離0。

合法tight根樹提供成本d[v]的真路，所以δ(s,v)≤d[v]；所有輸入arc的不等式d[v]≤d[u]+w沿任意walk累加，給d[v]≤任何walk成本，故d[v]≤δ(s,v)。兩向一起才相等。

若只驗不等式，圖sa1卻交d[s]=0,d[a]=−100會通過−100≤1，但沒有實現路徑；根樹tight檢查拒絕。若只驗tight根樹，圖sa5、sb1、ba1，選parent[a]=s、parent[b]=s、d=(0,5,1)都tight且有根；但ba要求5≤1+1不成立，真正a=2。

結論與檢核

距離相等的零權arc並不保證前驅方向無環。一般relax歷史的無環證明用最後更新時間；外部輸出必明查根連通性。

24-E07 邊數 DP 的負環重建

推導

以(s,a,b)排序：D_0=(0,∞,∞)、D_1=(0,1,∞)、D_2=(0,1,−2)、D_3=(0,−1,−2)。a在第3層嚴格改善，回溯$a_3 \leftarrow b_2 \leftarrow a_1 \leftarrow s_0$，得到s→a→b→a，成本1−3+1=−1。重複a截出a→b→a，總權−2，s可達。

一般若回溯walk少於n邊，就也是第n−1層候選，與$D_n[v] < D_{n-1}[v]$矛盾。恰n邊walk必重點，但可能有正、零、負多個cycles；必須分解後至少一個負。若全非負，刪掉全部cycles留下簡單路，邊數≤n−1且成本≤原值，再次矛盾。只說「含cycle」不足以判斷符號。

結論與檢核

完整n層×n點前驅表是$O(n^2)$輔助空間，另含輸入O(m)。兩層距離只用O(n)卻沒有本回溯法所需歷史；若要同時省空間須另給演算法與證明，不能直接省掉記錄仍宣稱可回溯。

第24章 單源最短路徑：詳解 （續）

24-E08 選擇算法及成本的必要條件

推導

(a) BFS Θ(n+m)，成本就是邊數；(b) 拓撲DAG Θ(n+m)，負邊可有；(c) Dijkstra，非負權下extract可固定；(d) BF偵測可達負cycle，若要完整距離語義則以第n層seed向前標$-\infty$，不把下降中的有限d交成答案。

給定n,m，BF主要$nm=3\times10^8$邊掃描。Binary為$(n+m)\log_2 n\approx4\times10^4\times13.288\approx5.32\times10^5$操作量級；Fibonacci為$m+n\log_2 n\approx1.63\times10^5$攤銷量級；線性掃key為$n^2+m\approx1.0003\times10^8$。不同基本操作常數與表示法不同，這些不是實測時間比。

結論與檢核

m=0時仍須初始化並交付n項，至少Ω(n)，BF為O(n+nm)。差分限制全零版本亦O(n+nm)；真正建super-source會新增n條邊，原m=0不代表增廣圖無邊。

原創驗算程式附錄: dijkstra_paths.py

```
"""Dijkstra on a nonnegative weighted graph."""

from heapq import heappop, heappush

def dijkstra(graph, source):
    distance = {v: float("inf") for v in graph}
    distance[source] = 0
    queue = [(0, source)]
    while queue:
        current, u = heappop(queue)
        if current != distance[u]:
            continue
        for v, weight in graph[u].items():
            candidate = current + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heappush(queue, (candidate, v))
    return distance


graph = {"s": {"a": 2, "b": 5}, "a": {"b": 1}, "b": {}}
assert dijkstra(graph, "s") == {"s": 0, "a": 2, "b": 3}
print(dijkstra(graph, "s"))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

原創驗算程式附錄: shortest_path_checks.py

```
"""Check Bellman-Ford, DAG relaxation, Dijkstra, and difference constraints."""

from math import inf
import heapq
import random


def bellman_ford(vertices, edges, source):
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    for _ in range(len(vertices) - 1):
        for u, v, weight in edges:
            if distance[u] != inf and distance[v] > distance[u] + weight:
                distance[v] = distance[u] + weight
    negative_cycle = any(
        distance[u] != inf and distance[v] > distance[u] + weight
        for u, v, weight in edges
    )
    return distance, negative_cycle


def dijkstra(vertices, adjacency, source):
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    queue = [(0, source)]
    visited = set()
    while queue:
        current, u = heapq.heappop(queue)
        if current != distance[u]:
            continue
        if u in visited:
            continue
        visited.add(u)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

```
        for v, weight in adjacency.get(u, ()):
            candidate = current + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heapq.heappush(queue, (candidate, v))
    return distance


def dag_shortest_paths(vertices, edges, source):
    adjacency = {vertex: [] for vertex in vertices}
    indegree = {vertex: 0 for vertex in vertices}
    for u, v, weight in edges:
        adjacency[u].append((v, weight))
        indegree[v] += 1
    ready = [vertex for vertex in vertices if indegree[vertex] == 0]
    order = []
    while ready:
        u = ready.pop()
        order.append(u)
        for v, _ in adjacency[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                ready.append(v)
    assert len(order) == len(vertices)
    distance = {vertex: inf for vertex in vertices}
    distance[source] = 0
    for u in order:
        if distance[u] == inf:
            continue
        for v, weight in adjacency[u]:
            distance[v] = min(distance[v], distance[u] + weight)
    return distance
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

```
def run():
    sample_edges = [("s", "a", 4), ("s", "b", 5), ("a", "b", -3)]
    distance, has_negative_cycle = bellman_ford({"s", "a", "b"}, sample_edges, "s")
    assert distance == {"s": 0, "a": 4, "b": 1}
    assert not has_negative_cycle

    cycle_edges = [("s", "a", 1), ("a", "b", -3), ("b", "a", 1)]
    _, has_negative_cycle = bellman_ford({"s", "a", "b"}, cycle_edges, "s")
    assert has_negative_cycle

    dag = dag_shortest_paths(
        {"s", "a", "b"},
        [("s", "a", 3), ("s", "b", 8), ("a", "b", -2)],
        "s",
    )
    assert dag == {"s": 0, "a": 3, "b": 1}

    adjacency = {"s": [("a", 2), ("b", 5)], "a": [("b", 1)]}
    assert dijkstra({"s", "a", "b"}, adjacency, "s") == {"s": 0, "a": 2, "b": 3}

    difference_solution = {"a": 2, "b": 0, "s": 0}
    assert difference_solution["a"] - difference_solution["b"] <= 2
    assert difference_solution["b"] - difference_solution["s"] <= 5

    negative_edge = {
        "s": [("a", 2), ("b", 1)],
        "a": [("b", -2)],
        "b": [("c", 2)],
    }
    wrong = dijkstra({"s", "a", "b", "c"}, negative_edge, "s")
    assert wrong["c"] == 3
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

```
    exact, _ = bellman_ford(
        {"s", "a", "b", "c"},
        [("s", "a", 2), ("s", "b", 1), ("a", "b", -2), ("b", "c", 2)],
        "s",
    )
    assert exact["c"] == 2

    rng = random.Random(24)
    cases = 0
    for size in range(2, 8):
        names = [f"v{i}" for i in range(size)]
        for _ in range(20):
            edges = []
            for i in range(size):
                for j in range(i + 1, size):
                    if rng.random() < 0.4:
                        edges.append((names[i], names[j], rng.randint(-5, 9)))
            bellman = bellman_ford(names, edges, names[0])[0]
            dag = dag_shortest_paths(names, edges, names[0])
            assert bellman == dag
            cases += 1

    print(
        f"PASS Bellman-Ford_negative_cycle=yes DAG_cases={cases} "
        "Dijkstra_negative_edge_failure=yes difference_constraints=yes"
    )


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

原創驗算程式附錄: sssp_depth_checks.py

```
"""Finite SSSP checks against enumerated simple paths/cycles, not proof substitutes."""
from math import inf
from itertools import product
from random import Random
from collections import deque

SAMPLE=[(3,4,3),(1,4,8),(1,3,1),(2,4,12),(2,3,5),(2,1,-2),(0,1,4),(0,2,1)]
POSITIVE=[(u,v,2 if (u,v)==(2,1) else w) for u,v,w in SAMPLE]


def reachable(n,edges,sources):
    adj=[[] for _ in range(n)]
    for u,v,_ in edges:adj[u].append(v)
    seen=set(sources);q=deque(sources)
    while q:
        for v in adj[q.popleft()]:
            if v not in seen:seen.add(v);q.append(v)
    return seen


def oracle(n,edges,s):
    adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):adj[u].append((v,w,i))
    d=[inf]*n;cycles=[]
    def paths(u,seen,cost):
        d[u]=min(d[u],cost)
        for v,w,_ in adj[u]:
            if v not in seen:paths(v,seen|{v},cost+w)
    paths(s,{s},0)
    for start in range(n):
        def visit(u,seen,cost,ids):
            for v,w,i in adj[u]:
                if v==start and cost+w<0:cycles.append((set(seen),ids+[i]))
                elif v not in seen:visit(v,seen|{v},cost+w,ids+[i])
        visit(start,{start},0,[])
    r=reachable(n,edges,[s]);seeds=set().union(*(vs for vs,_ in cycles if vs&r)) if cycles
else set()
    affected=reachable(n,edges,seeds)
    for v in affected:d[v]=-inf
    return d


def bellman(n,edges,s,trace=False):
    d=[inf]*n;pi=[None]*n;d[s]=0;stages=[d[:]]
    for _ in range(n-1):
        changed=False
        for i,(u,v,w) in enumerate(edges):
            if d[u]!=inf and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i;changed=True
        stages.append(d[:])
        if not changed:break
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

```
    bad=[i for i,(u,v,w) in enumerate(edges) if d[u]!=inf and d[u]+w<d[v]]
    return d,pi,bad,stages


def budget(n,edges,s):
    d=[inf]*n;d[s]=0;layers=[d[:]];parents=[]
    for _ in range(n):
        old=d;d=old[:];p=[None]*n
        for i,(u,v,w) in enumerate(edges):
            if old[u]!=inf and old[u]+w<d[v]:d[v]=old[u]+w;p[v]=i
        layers.append(d);parents.append(p)
    seeds={v for v in range(n) if layers[n][v]<layers[n-1][v]}
    affected=reachable(n,edges,seeds)
    result=layers[n-1][:]
    for v in affected:result[v]=-inf
    return result,layers,parents,seeds


def negative_witness(n,edges,s):
    _,layers,parents,seeds=budget(n,edges,s)
    if not seeds:return None
    v=min(seeds);back=[]
    for i in range(n,0,-1):
        identity=parents[i-1][v]
        if identity is not None:back.append(identity);v=edges[identity][0]
    walk=list(reversed(back));assert v==s and len(walk)==n
    vertices=[s];stack_edges=[];where={s:0};u=s
    for identity in walk:
        a,b,w=edges[identity];assert a==u;u=b
        if b in where:
            j=where[b];cycle=stack_edges[j:]+[identity]
            if sum(edges[k][2] for k in cycle)<0:return cycle
            for x in vertices[j+1:]:del where[x]
            vertices=vertices[:j+1];stack_edges=stack_edges[:j]
        else:
            stack_edges.append(identity);where[b]=len(vertices);vertices.append(b)
    raise AssertionError('strict nth budget improvement must contain a negative cycle')


def dense_dijkstra(n,edges,s):
    assert all(w>=0 for _,_,w in edges)
    adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):adj[u].append((v,w,i))
    d=[inf]*n;pi=[None]*n;d[s]=0;pending=set(range(n));trace=[]
    while pending:
        u=min(pending,key=lambda v:(d[v],v));pending.remove(u)
        if d[u]==inf:break
        for v,w,i in adj[u]:
```

第24章 單源最短路徑：詳解 （續）

```
            if v in pending and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i
        trace.append((u,d[u],d[:],pi[:]))
    return d,pi,trace


def dag(n,edges,s):
    indegree=[0]*n;adj=[[] for _ in range(n)]
    for i,(u,v,w) in enumerate(edges):indegree[v]+=1;adj[u].append((v,w,i))
    q=deque(v for v in range(n) if indegree[v]==0);order=[]
    while q:
        u=q.popleft();order.append(u)
        for v,w,i in adj[u]:
            indegree[v]-=1
            if indegree[v]==0:q.append(v)
    if len(order)!=n:return None
    d=[inf]*n;pi=[None]*n;d[s]=0
    for u in order:
        for v,w,i in adj[u]:
            if d[u]!=inf and d[u]+w<d[v]:d[v]=d[u]+w;pi[v]=i
    return d,pi


def certificate(n,edges,s,d,pi):
    r=reachable(n,edges,[s])
    if d[s]!=0 or pi[s] is not None:return False
    children=[[] for _ in range(n)]
    for v in range(n):
        if v not in r:
            if d[v]!=inf or pi[v] is not None:return False
        else:
            if d[v] in (inf,-inf):return False
            if v!=s:
                i=pi[v]
                if i is None or not 0<=i<len(edges):return False
                u,to,w=edges[i]
                if to!=v or d[v]!=d[u]+w:return False
                children[u].append(v)
    seen={s};q=deque([s])
    while q:
        for v in children[q.popleft()]:
            if v in seen:return False
            seen.add(v);q.append(v)
    return seen==r and all(d[v]<=d[u]+w for u,v,w in edges if u in r)


def verify(n,edges,s):
    expected=oracle(n,edges,s);actual,layers,parents,seeds=budget(n,edges,s)
    assert actual==expected
    d,pi,bad,_=bellman(n,edges,s)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第24章 單源最短路徑：詳解 （續）

```
    assert bool(bad)==any(v==-inf for v in expected)
    if not bad:assert d==expected and certificate(n,edges,s,d,pi)
    witness=negative_witness(n,edges,s)
    if bad:
        assert witness and sum(edges[i][2] for i in witness)<0
        assert all(edges[witness[i]][1]==edges[witness[(i+1)%len(witness)]][0] for i in
range(len(witness)))
        assert edges[witness[0]][0] in reachable(n,edges,[s])
    else:assert witness is None
    acyclic=dag(n,edges,s)
    if acyclic is not None:assert acyclic[0]==expected and certificate(n,edges,s,*acyclic)
    if all(w>=0 for _,_,w in edges):
        out=dense_dijkstra(n,edges,s);assert out[0]==expected and
certificate(n,edges,s,*out[:2])
    return bad


def main():
    rng=Random(24024);cases=sources=0
    for n in range(1,4):
        slots=[(u,v) for u in range(n) for v in range(n)]
        for values in product((None,-1,2),repeat=len(slots)):
            edges=[(u,v,w) for (u,v),w in zip(slots,values) if w is not None]
            for s in range(n):verify(n,edges,s);sources+=1
            cases+=1
    for _ in range(300):
        n=rng.randrange(4,7);edges=[(u,v,rng.randrange(-3,7)) for u in range(n) for v in
range(n) if rng.random()<.22]
        verify(n,edges,rng.randrange(n));cases+=1;sources+=1
    d,pi,bad,stages=bellman(6,SAMPLE,0)
    assert d==[0,-1,1,0,3,inf] and not bad
    assert stages==[[0,inf,inf,inf,inf,inf],[0,4,1,inf,inf,inf],[0,-1,1,5,12,inf],
[0,-1,1,0,7,inf],[0,-1,1,0,3,inf],[0,-1,1,0,3,inf]]
    pos=dense_dijkstra(6,POSITIVE,0)
    assert [(u,v) for u,v,*_ in pos[2]]==[(0,0),(2,1),(1,3),(3,4),(4,7)]
    neg=SAMPLE+[(3,1,-2)];assert budget(6,neg,0)[0]==[0,-inf,1,-inf,-inf,inf]
    zero=[(0,1,0),(1,2,0),(2,1,0)];dd,pp,_,_=bellman(3,zero,0)
    assert certificate(3,zero,0,dd,pp)
    assert not certificate(3,zero,0,dd,[None,2,1])
    constraints=[(0,1,-2),(1,2,-1),(2,0,4)]
    augmented=constraints+[(3,v,0) for v in range(3)]
    assert bellman(4,augmented,3)[0]==[0,-2,-3,0]
    assert bellman(4,[(0,1,-2),(1,2,-1),(2,0,2)]+[(3,v,0) for v in range(3)],3)[2]
    print(f'PASS weighted_graphs={cases} source_cases={sources} simple_path_cycle_oracle=True
'
          'nativeBF=budget_classification=DAG=Dijkstra_on_domains=True '
          'every_negative_witness_reachable_closed_strictly_negative=True
zero_cycle_parent_rejected=True '
          'sample_BF=0,-1,1,0,3,inf Dijkstra_order=s,b,a,c,t difference_constraints=True')


if __name__=='__main__':main()
```

第24章 單源最短路徑：詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study